



Unidad 2

Clase 3

Distintos modelos NoSQL

Modelo documental y consultas NoSQL

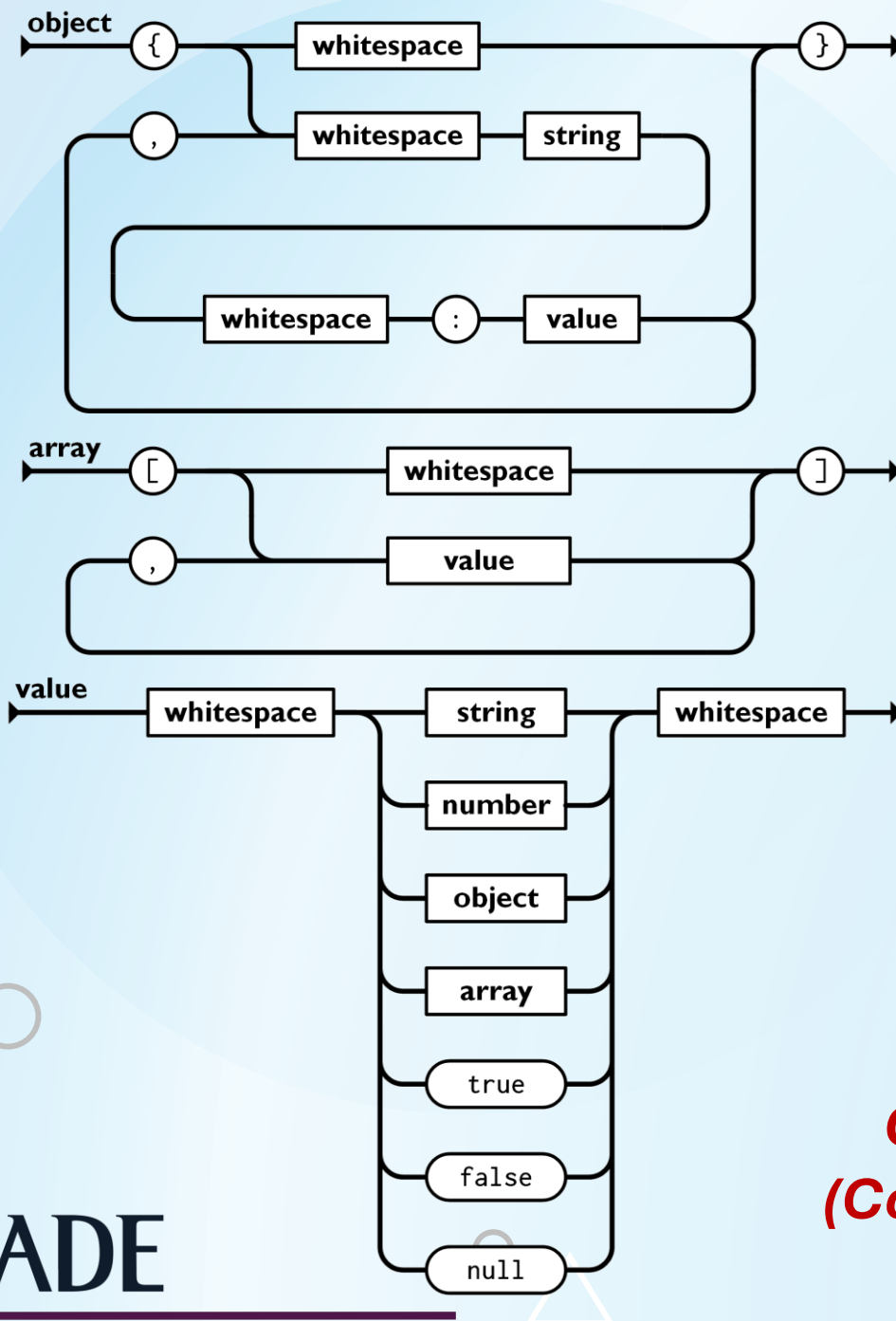
Diseñar y consultar estructuras documentales utilizando MongoDB, comprendiendo el modelo documental, la desnormalización y las operaciones básicas de acceso y recuperación de información.

JSON (JavaScript Object Notation)



JSON es un formato de texto ligero e independiente del lenguaje para el intercambio de datos estructurados. Basado en pares clave-valor y arreglos, es fácil de leer para humanos y rápido de interpretar por máquinas, siendo estándar en APIs web y configuraciones, especialmente para comunicación entre aplicaciones.

En un archivo JSON, los valores pueden ser Cadenas (String), Números, Booleanos, Objetos (Conjuntos de pares clave-valor), Arrays (Listas de valores encerradas entre corchetes), Null.



¿QUÉ ES JSON?

JavaScript Object Notation (JSON) es un **estándar para el intercambio de información** que usa la sintaxis de objetos de JavaScript.

```
1 {  
2   "title"    : "Person",  
3   "id"       : 1,  
4   "active"   : true,  
5   "properties" : {  
6     "name"    : "Beto",  
7     "company" : "EDteam",  
8     "courses" : ["PHP", "React"]  
9   }  
10 }
```

Todo objeto va encerrado entre llaves (**líneas 1 y 10**)

Parejas **clave: valor** separadas por comas.

Los valores pueden ser **strings, números, booleans, objetos, arrays o nulos.**

Las claves (llamadas también atributos o propiedades) **son siempre strings (entrecomillados).**

USOS

En API REST o GraphQL

Comunicaciones MQTT, SPI y Serial.

Interacciones AJAX o WebSockets

USO

Guardar configuraciones de programas

Intercambiar datos entre sistemas (APIs)

Almacenar información de forma organizada (catálogo de productos, precios..)

Portabilidad de datos

Objeto en memoria para interactuar con programas.

Estructura de Documento JSON

```
{
  "pedidoId": 1001,
  "cliente": {
    "nombre": "Ana López",
    "ciudad": "Morón"
  },
  "productos": [
    { "nombre": "Teclado",
      "cantidad": 1,
      "precio": 250000 },
    { "nombre": "Mouse",
      "cantidad": 2,
      "precio": 120000 }
  ],
  "estado": "en preparación"
}
```

Datos Embebidos

Lista de Productos



Documentos Individuales



Colección de Documentos



Base de Datos MongoDB

Desnormalización

Datos relacionados en un solo documento



Flexibilidad

Esquema adaptable y dinámico



BASE DE DATOS DOCUMENTALES



Una base de datos documental es un sistema NoSQL diseñado para almacenar, recuperar y gestionar información semiestructurada, típicamente en formatos JSON o BSON. A diferencia de las relacionales, no requieren esquemas rígidos, permitiendo flexibilidad en los datos y un alto rendimiento para aplicaciones modernas. Son ideales para Big Data, catálogos, sistemas de gestión de contenidos y perfiles de usuario.

Cuando usar base documental?



cuando la flexibilidad y la velocidad de desarrollo son más importantes que la rigidez de una tabla

1. Cuando tus datos cambian seguido (Esquema flexible) Si estás lanzando un prototipo o una Startup y no sabés qué datos vas a necesitar mañana. Ejemplo: Un catálogo de productos de e-commerce. Un celular tiene "memoria RAM", pero una remera tiene "talle" y "color". En JSON, guardás cada uno con sus propias claves sin romper nada.

Cuando usar base documental?



cuando la flexibilidad y la velocidad de desarrollo son más importantes que la rigidez de una tabla

2. Contenido Jerárquico o "Anidado":

Cuando la información está muy relacionada entre sí y querés evitar los famosos "Joins" (cruces de tablas) de las bases tradicionales. *Ejemplo: Un post de un blog. En un solo JSON guardás el título, el autor, los comentarios y los tags. Al pedir el post, traés todo de un solo golpe.*

3. Aplicaciones de Tiempo Real y Grandes Volúmenes: Son excelentes para manejar muchísimos datos que entran rápido y no son del todo uniformes. *Ejemplo: Apps de Internet de las Cosas (IoT) que reciben datos de sensores, o el muro de noticias de una Red Social.*

Cuando usar base documental?



cuando la flexibilidad y la velocidad de desarrollo son más importantes que la rigidez de una tabla

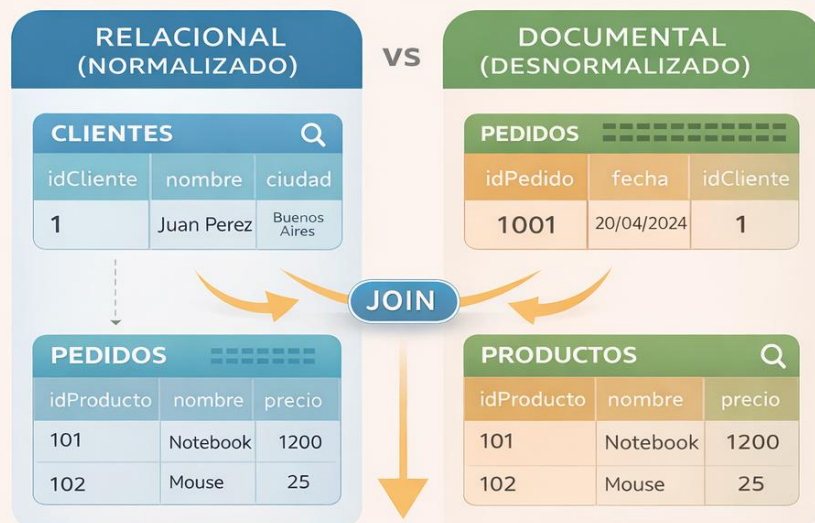
4. Cuando programás en JavaScript (o Node.js). Si tu lenguaje de programación ya usa JSON de forma nativa, usar una base de datos documental (como MongoDB) es como hablar el mismo idioma en ambos lados. No necesitás "traductores" pesados entre tu código y tu base de datos.

5. Gestión de Configuraciones y Perfiles: Para guardar preferencias de usuarios o ajustes de aplicaciones donde cada usuario puede tener configuraciones distintas y únicas.

Desnormalización



La desnormalización en bases de datos documentales (como MongoDB) es una técnica de modelado que integra datos relacionados dentro de un mismo documento (embedding) para evitar uniones (joins) complejas. A diferencia de los sistemas relacionales, aquí se prioriza la velocidad de lectura y la eficiencia de las consultas sobre la integridad referencial, duplicando información estratégicamente.



DOCUMENTO DESNORMALIZADO

```

PEDIDO {
  pedido: 1001,
  cliente: { "Juan Perez",
    ciudad: { "Buenos Aires"
  },
  productos: [
    { nombre: "Notebook",
      precio: 1200,
    },
    { nombre: "Mouse",
      precio: 25
    }
  ],
  total: 1225
}
  
```

- ✓ Datos redundantes
- ✓ Consulta más rápida



Actividad

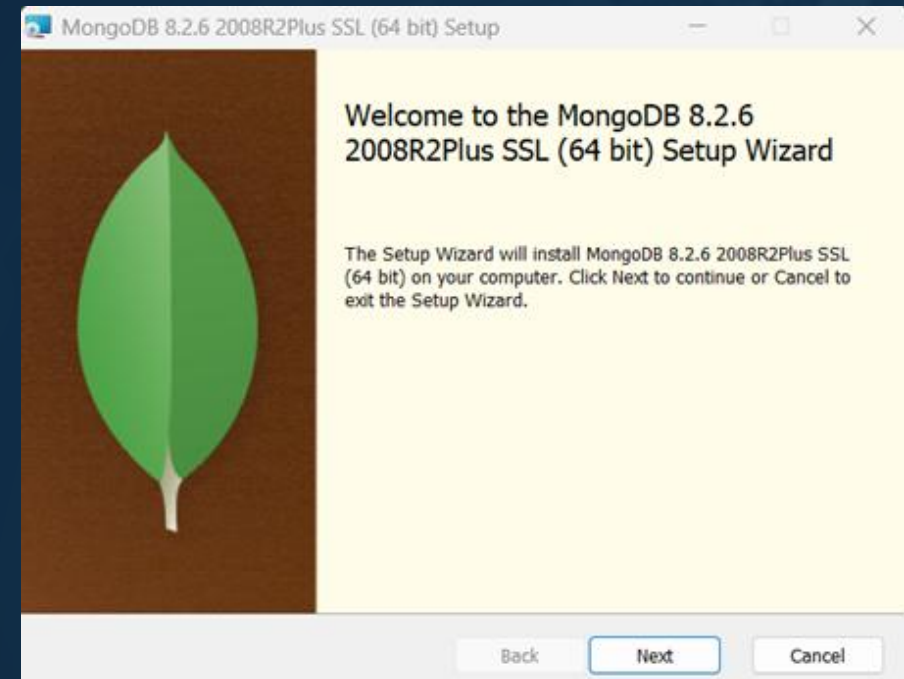
Instalación MongoDB

<https://www.mongodb.com/try/download/community>

Instalar MongoDB

Instalar consola mongosh

MongoDB Atlas (cloud – on macOS)





```
Current Mongosh Log ID: 69c07f711f464bbddf114029
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.8.1
Using MongoDB:      8.2.6
Using Mongosh:       2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
  2026-03-22T20:21:58.348-03:00: Access control is not enabled for the database. Read and write access to data and conf
  igation is unrestricted
  -----

test>
```




```
test> show dbs
admin      40.00 KiB
clase03    56.00 KiB
config     72.00 KiB
local      40.00 KiB
test>
```

```
test> use Clase03
switched to db Clase03
Clase03>
```

```
clase03> db.alumnos.insertOne({
|   nombre: "Juan",
|   materia: "Ingeniería de Datos II",
|   modelo: "Relacional"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69c09a065439d12d7211402a')
}
clase03>
```

<https://www.mongodb.com/es/docs/v8.0/reference/command/>

`show dbs`: Muestra todas las bases de datos disponibles.

`use <nombre db>`: Cambia a una base de datos específica; si no existe, la crea al insertar datos.

`db`: Muestra la base de datos actual en uso.

`show collections`: Muestra las colecciones de la base de datos actual.

`db.createCollection("<nombre>")`: Crea una nueva colección explícitamente.

`db.dropDatabase()`: Elimina la base de datos actual.

`db.<coleccion>.drop()`: Elimina una colección específica

`help`: Muestra la ayuda general del shell.

`db.help()`: Muestra ayuda para los métodos de la base de datos.

`db.collection.help()`: Muestra ayuda para los métodos de colecciones.

`cls`: Limpia la pantalla.



ESTRUCTURA CONCEPTUAL DE MONGO DB



BASE DE DATOS

Contenedor principal

clase03



COLECCIONES

Conjunto de documentos similares

alumnos

materias

materias



DOCUMENTOS

Registros en formato JSON

JSON

✓ Flexible

```
{
  "nombre": "Ana",
  "materia": "Ingeniería de Datos II",
  "legajo": 112345
}
```

_____ { clave: valor }

JSON



OBJETOS EMBEBIDOS

Objetos dentro de objetos

JSON

```
"contacto": {
  "email": "ana@mail.com",
  "ciudad": "Morón"
}
```

Lista de objetos



ARRAYS

Listas de valores

Lista de objetos

ARRAYS

```
"materias": [
  { "Ingeniería de Datos II",
    "Programación"
}
```

_____ (Inter CSON -) -

MongoDB guarda información como documentos JSON organizados en colecciones dentro de una base de datos, permitiendo estructuras anidadas y listas sin necesidad de definir un esquema previo.

Creación de base y colección

```
mongosh mongodb://127.0.0.1
test> use agendaInteractiva
switched to db agendaInteractiva
agendaInteractiva> db.contactos.insertOne({
  nombre: "María López",
  telefono: "11-5555-1234",
  email: "maria@gmail.com",
  ciudad: "Morón"
})
{
  acknowledged: true,
  insertedId: ObjectId('69c0a075212e51baa411402a')
}
agendaInteractiva> show collections
contactos
agendaInteractiva> db.contactos.find().pretty()
[
  {
    _id: ObjectId('69c0a075212e51baa411402a'),
    nombre: 'María López',
    telefono: '11-5555-1234',
    email: 'maria@gmail.com',
    ciudad: 'Morón'
  }
]
agendaInteractiva>
```

```
agendaInteractiva> db.contactos.insertOne({
  nombre: "Carlos Gómez",
  telefono: "11-4444-9876",
  email: "carlos@gmail.com",
  direccion: {
    calle: "Rivadavia 123",
    ciudad: "Morón",
    provincia: "Buenos Aires"
  },
  etiquetas: ["trabajo", "cliente", "urgente"]
})
{
  acknowledged: true,
  insertedId: ObjectId('69c0a133212e51baa411402b')
}
agendaInteractiva>
```

Campos
simples

Objeto
embebido

Array

Inserción de documentos con estructura anidada

```
mongosh mongodb://127.0.0.1:27027/agendaInteractiva> db.contactos.insertOne({
  nombre: "María López",
  telefono: "11-5555-1234"
})
{
  acknowledged: true,
  insertedId: ObjectId('69c0a2c7212e51baa411402c')
}
agendaInteractiva> db.contactos.insertOne({
  nombre: "Carlos Gómez",
  email: "carlos@gmail.com",
  direccion: {
    calle: "Rivadavia 123",
    ciudad: "Morón",
    provincia: "Buenos Aires"
  },
  telefonos: [
    { tipo: "personal", numero: "11-4444-1111" },
    { tipo: "trabajo", numero: "11-4444-2222" }
  ],
  etiquetas: ["cliente", "urgente", "proveedor"]
})
{
  acknowledged: true,
  insertedId: ObjectId('69c0a2e4212e51baa411402d')
}
agendaInteractiva>
```

```
mongosh mongodb://127.0.0.1:27027/agendaInteractiva> db.contactos.find().pretty()
[
  {
    _id: ObjectId('69c0a075212e51baa411402a'),
    nombre: 'María López',
    telefono: '11-5555-1234',
    email: 'maria@gmail.com',
    ciudad: 'Morón'
  },
  {
    _id: ObjectId('69c0a133212e51baa411402b'),
    nombre: 'Carlos Gómez',
    telefono: '11-4444-9876',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    etiquetas: [ 'trabajo', 'cliente', 'urgente' ]
  },
  {
    _id: ObjectId('69c0a2c7212e51baa411402c'),
    nombre: 'María López',
    telefono: '11-5555-1234'
  },
  {
    _id: ObjectId('69c0a2e4212e51baa411402d'),
    nombre: 'Carlos Gómez',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    telefonos: [
      { tipo: 'personal', numero: '11-4444-1111' },
      { tipo: 'trabajo', numero: '11-4444-2222' }
    ],
    etiquetas: [ 'cliente', 'urgente', 'proveedor' ]
  }
]
agendaInteractiva>
```

Inserción de documentos con estructura anidada

```
mongosh mongodb://127.0.0.1
agendaInteractiva> db.contactos.find({ "direccion.ciudad": "Morón" })
[
  {
    _id: ObjectId('69c0a133212e51baa411402b'),
    nombre: 'Carlos Gómez',
    telefono: '11-4444-9876',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    etiquetas: [ 'trabajo', 'cliente', 'urgente' ]
  },
  {
    _id: ObjectId('69c0a2e4212e51baa411402d'),
    nombre: 'Carlos Gómez',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    telefonos: [
      { tipo: 'personal', numero: '11-4444-1111' },
      { tipo: 'trabajo', numero: '11-4444-2222' }
    ],
    etiquetas: [ 'cliente', 'urgente', 'proveedor' ]
  }
]
agendaInteractiva> |
```

```
mongosh mongodb://127.0.0.1
agendaInteractiva> db.contactos.find({ etiquetas: "proveedor" })
[
  {
    _id: ObjectId('69c0a2e4212e51baa411402d'),
    nombre: 'Carlos Gómez',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    telefonos: [
      { tipo: 'personal', numero: '11-4444-1111' },
      { tipo: 'trabajo', numero: '11-4444-2222' }
    ],
    etiquetas: [ 'cliente', 'urgente', 'proveedor' ]
  }
]
agendaInteractiva>
```

Consultas simples para recuperar datos

- `db.contactos.find()`
Esto devuelve todo el contenido. Es la consulta más básica y compacta.
- Si querés verlo más legible con indentación y saltos
`db.contactos.find().pretty()`
- Consultar por un campo simple:
`db.contactos.find({ nombre: "Carlos Gómez" })`
MongoDB busca documentos donde ese campo coincida.
- Recuperar por un campo dentro de un objeto anidado:
`db.contactos.find({ "direccion.ciudad": "Morón" })`
- Recuperar dentro de un array simple:
`db.contactos.find({ etiquetas: "cliente" })`
MongoDB busca automáticamente dentro del array.
- Recuperar dentro de un array de objetos:
`db.contactos.find({ "telefonos.tipo": "trabajo" })`

Actualización de documentos – Método **updateOne** y Operador **\$set**

```
mongosh mongodb://127.0.0.1 > db.contactos.updateOne(
  { nombre: "Carlos Gómez" },
  { $set: { ciudad: "Castelar" } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
mongosh mongodb://127.0.0.1 > db.contactos.updateOne(
  { nombre: "Carlos Gómez" },
  { $set: { legajo: 112345 } }
)
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
mongosh mongodb://127.0.0.1 >
```

```
mongosh mongodb://127.0.0.1 > db.contactos.find({ nombre: "Carlos Gómez" })
[
  {
    _id: ObjectId('69c0a133212e51baa411402b'),
    nombre: 'Carlos Gómez',
    telefono: '11-4444-9876',
    email: 'carlos@gmail.com',
    direccion: { calle: 'Rivadavia 123', ciudad: 'Morón', provincia: 'Buenos Aires' },
    etiquetas: [ 'trabajo', 'cliente', 'urgente' ],
    ciudad: 'Castelar',
    legajo: 112345
  },
]
```

Eliminación de registros

`deleteOne` → elimina un documento

`deleteMany` → elimina varios documentos

filtro `{ }` → elimina todos los registros

`drop()` → elimina la colección completa

```
db.contactos.deleteOne({ nombre: "Carlos Gómez" })
```

MongoDB busca el primer documento que cumpla la condición y lo elimina.

```
db.contactos.deleteMany({ etiquetas: "cliente" })
```

Esto borra todos los documentos que coincidan.

```
db.contactos.deleteMany({ })
```

MongoDB elimina **todos los documentos** de la colección, porque el filtro vacío coincide con todos. Esto no borra la colección, solo su contenido.

```
db.contactos.drop()
```

Eso sí elimina la colección entera.

Eliminación de registros seleccionados

`deleteOne()` siempre elimina **el primer documento que coincide con el filtro**.

Para seleccionar otro documento coincidente en el filtro, se debe **identificar el documento de forma única** y borrar por ese identificador. Cada documento tiene un `_id`.

Primero se busca:

```
db.contactos.find({ ciudad: "Morón" })
```

Tomás el `_id` del segundo documento y luego borrás:

```
db.contactos.deleteOne({ _id: ObjectId("...") })
```

Eso es lo más preciso porque cada documento tiene un identificador único.



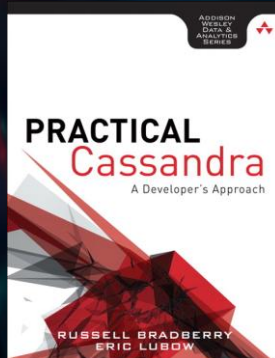
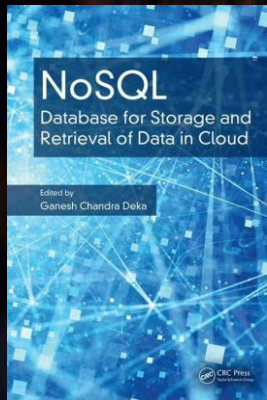
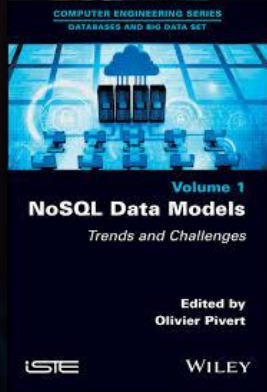
Actividad

Base documental de reuniones de trabajo

La empresa necesita registrar reuniones internas de distintas áreas para conservar información sobre temas tratados, participantes, decisiones tomadas y próximos encuentros.

El objetivo es construir una base documental en MongoDB que permita almacenar cada reunión como un documento y luego consultar reuniones por tema o motivo.





BIBLIOGRAFIA



MongoDB — Documentación oficial: “Documents and Data Model”. Explica la estructura de documentos, objetos embebidos y arrays.

<https://www.mongodb.com/docs/manual/core/document/>

MongoDB — “CRUD Operations”. Incluye insertOne, find, updateOne y deleteOne.

<https://www.mongodb.com/docs/manual/crud/>

MongoDB — “Query Documents”. Consultas simples sobre campos y estructuras anidadas.

<https://www.mongodb.com/docs/manual/tutorial/query-documents/>



PROXIMA CLASE



Clase 4 – Diseño documental y analítica de datos aplicada

Tema:

MongoDB / MQL avanzado

- Agregaciones
- Índices
- Diseño orientado a consultas
- Casos de auditoría (hallazgos, evidencia, eventos)

Ejercitación aplicada al TP.